Answer Box #1

What are 2 reasons you may have to use a class instead of a record?

mutable fields

participate in inheritance

records cannot extend or be extended

#2

What are abstract classes good for that interfaces are not?

sharing fields across multiple classes

```
abstract class A {
    String message;
    abstract void printMessage();
}

class D extends C { }

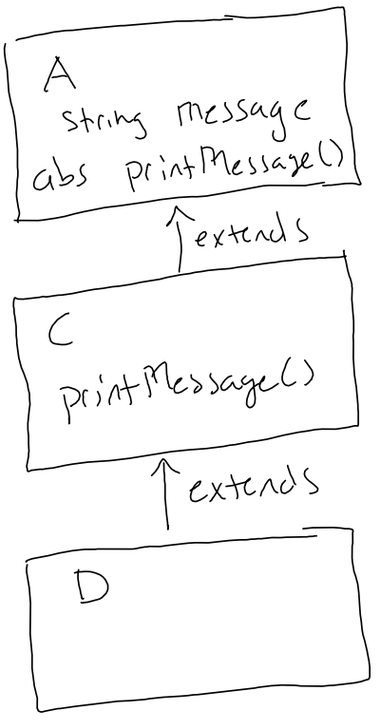
new D().printMessage()
```

```
class C extends A {
    @Override
    void printMessage() {
        Sys.out.pl(this.message);
    }
}
```

```
class E extends A { }
```

X E does not impl printMessage()

1. Does D have printMessage()? No, check parent
2. Does C have printMessage()? Yes, call it



Posts, Two Ways

Name: _____ Date: _____

Records or an abstract class?

Two versions of the same three kinds of post. Both start with `interface Post { String preview(); }`.

Version R: records

```
record TextPost(String user, String message,
    boolean edited) implements Post {
  public String preview() {
    if (edited) {
      return "@" + user + ": " + message
          + " (edited)";
    }
    return "@" + user + ": " + message;
  }
}
record VideoPost(String user, String message,
    int seconds) implements Post {
  public String preview() {
    return "@" + user + ": " + message
        + " [" + seconds + "s]";
  }
}
record Repost(String user, String message,
    String from) implements Post {
  public String preview() {
    return "@" + user + " reposted @" + from
        + ": " + message;
  }
}
```

Version A: an abstract class

```
abstract class Message implements Post {
  String user;
  String message;
  Message(String user, String message) {
    this.user = user;
    this.message = message;
  }
}
class TextPost extends Message {
  boolean edited;
  TextPost(String user, String message,
      boolean edited) {
    super(user, message);
    this.edited = edited;
  }
  public String preview() { /* same as R */ }
}
class VideoPost extends Message {
  int seconds;
  VideoPost(String user, String message,
      int seconds) {
    super(user, message);
    this.seconds = seconds;
  }
  public String preview() { /* same as R */ }
}
class Repost extends Message {
  String from;
  Repost(String user, String message,
      String from) {
    super(user, message);
    this.from = from;
  }
  public String preview() { /* same as R */ }
}
```

Q1. Before counting: which version would you rather have written? Why?

Answer Box #3 (say why!)

Q2. Written out in full, with each `preview` as in R and one statement per line, R is 17 lines and A is 40. Where do A's extra lines go?

Field declarations (boolean edited) and constructors "boilerplate"

Q3. Circle what this prints in each version, and say why they differ.

```
System.out.println(new TextPost("ana", "hi", false).equals(new TextPost("ana", "hi", false)));
```

R: (true) / false A: true / (false)

No overridden .equals on the classes

Q4. Every post now needs a `String date`, given when the post is made. In each version, which lines change? Predict which version needs fewer edits, then count.

R: add field to each record

A: Add String date to Message .... also change all constructors!

Q5. Posts now get `int likes`, starting at 0, and a method `void like()` that adds one. What has to change in each version?

R: Uh... rewrite all as classes?

A: Similar to date, but keep constrs and initialize to 0

field int likes
constr this.likes = 0

int likes() { return this.likes }
void like() { this.likes += 1 }

One class per kind, or one class with a `kind`?

Version K: a class per kind

```
interface Post {
  String preview();
  boolean isLong();
}
record TextPost(String user, String message)
    implements Post {
  public String preview() {
    return "@" + user + ": " + message;
  }
  public boolean isLong() {
    return message.length() > 80;
  }
}
record VideoPost(String user, String message,
    int seconds) implements Post {
  public String preview() {
    return "@" + user + ": " + message
        + " [" + seconds + "s]";
  }
  public boolean isLong() { return seconds > 60; }
}

Post a = new TextPost("ana", "Exam moved to Friday");
Post b = new VideoPost("ben", "My cat learns Java",
42);
```

Version F: one record with a `kind` field

```
record Post(String kind, String user,
    String message, int seconds) {
  String preview() {
    if (kind.equals("video")) {
      return "@" + user + ": " + message
          + " [" + seconds + "s]";
    }
    return "@" + user + ": " + message;
  }
  boolean isLong() {
    if (kind.equals("video")) {
      return seconds > 60;
    }
    return message.length() > 80;
  }
}

Post a = new Post("text", "ana",
    "Exam moved to Friday", 0);
Post b = new Post("video", "ben",
    "My cat learns Java", 42);
```

what type is it?

use an enum for kind

treated as text post!

Q6. In F, what does the `0` in `a` mean? What does `new Post("vidoe", "ben", "My cat learns Java", 42).preview()` return (note `"vidoe"`, a typo for `"video"`), and what would the same typo do in K?

0 is a placeholder for saying "no video"

Q7. Make each change in both versions, and count the places you touch. Count a new class, a new method, an edited line, or an edited `new` as one place each.

change	K: places	F: places
Add link posts: a `String url`, previewed as `"@ana: Read this <url>"`, never long.		
Add `boolean mentions(String word)`: true when the message contains `word`.		

Q8. A feed app adds a new kind of post every month, but rarely a new operation. Which version would you pick? Which if it is the other way around?

Tricks + hacks

A int[]
B Likes

```
class Likes { int likes; }
record TextPost(String user, String message, boolean edited, Likes likes) {
```

Plus a custom constructor to init likes

```
    //"increase" the likes counter by 1
    void like() {
        A this.likes.likes += 1;
        B this.likes[0] += 1;
    }
    int likes() { //return current count
        A return this.likes.likes;
        B return this.likes[0];
    }
```

More work to do in equals

```
}
```

TextPost p = new TextPost(...)

p @1001

@1001
TP["u", "m", likes: @1002]

@1002
Likes[likes=0]

Composition vs. Inheritance

Q4: Adding String date

just add date!

```
record Message(String user, String message) {
```

String date

```
    String previewPrefix() { return "@" + user + ": " + message; }
}
```

↑ helper method → use if useful!

```
record TextPost(Message m, boolean edited) implements Post {
    public String preview() {
        if(edited) { return m.previewPrefix() + "(edited)"; }
        return "@" + m.user + ": " + m.message;
    } return m.previewPrefix();
}
record VideoPost(Message m, int seconds) implements Post {
    public String preview() {
        return "@" + m.user + ": " + m.message + "[" + seconds + "]";
    } return m.previewPrefix() + "[" + seconds + "]";
}
```

should use m.message()

(Repost has a different preview format — would not use helper. That's fine!)